Dockerized Entertainment React Service

Course Name: DevOps

Institution Name: Medicaps University – Datagami Skill Based Course

Student Name(s) & Enrolment Number(s):

Sr no	Student Name	Enrolment Number
1	AMAN SHARMA	EN22CS301109
2	ANANYA BHATIA	EN22CS301117
3	ANKIT NAGAR	EN22CS301134
4	ANUSHA SINGH PANWAR	EN22CS301179
5	ARPIT PATIDAR	EN22CS301199

Group Name: Group 02D2

Project Number: DO-14

Industry Mentor Name: Mr. Vaibhav

University Mentor Name: Prof. Akshay Saxena

Academic Year: Jan-June 2026

Problem Statement & Objectives

1. Problem Statement

The process of generating structured entertainment content such as story ideas, loglines, outlines, and scene descriptions is complex and requires multiple stages of creative development. Each stage depends on maintaining logical flow, contextual consistency, and coherence across the entire content generation pipeline.

Traditional content creation methods are manual, time-consuming, and rely heavily on human creativity and effort. Users often face difficulty in transforming a simple idea into a well-structured output, especially when multiple stages of content generation are involved.

Although Generative AI has significantly improved content generation capabilities, most existing systems function as **single-step response generators**, producing outputs based on isolated prompts without maintaining continuity across stages. This leads to several limitations:

- Lack of structured multi-stage content generation
- Inability to maintain contextual continuity across different stages
- Absence of semantic memory for storing and retrieving previous outputs
- Inconsistent or unstructured content generation
- Limited reusability of generated information
- Dependency on manual effort to guide each stage

In addition to these challenges, deploying AI-based applications reliably across different systems is difficult due to:

- Dependency conflicts
- Environment configuration issues
- Lack of portability

These issues make it challenging to use such systems in real-world scenarios.

Therefore, there is a need for a system that:

- Supports structured multi-stage content generation
- Maintains contextual continuity across stages
- Utilizes memory for efficient retrieval of relevant information

- Produces coherent and structured outputs
- Ensures consistent and reliable deployment across environments

The problem addressed in this project is:

To design and implement a **Dockerized Generative AI-based entertainment content generation system** that enables structured, context-aware, and multi-stage content creation while ensuring portability, consistency, and ease of deployment through containerization.

2. Project Objectives

The primary objectives of this project are to develop a structured and deployable AI-based system for entertainment content generation while ensuring consistency, scalability, and ease of use.

The key objectives of the project are:

- To design a system capable of **multi-stage content generation** (concepts, loglines, outlines, etc.)
- To automate the transformation of a simple idea into **structured and meaningful content** using Generative AI
- To maintain **contextual continuity** across different stages using semantic memory (FAISS)
- To integrate **Large Language Models (LLMs)** for intelligent and high-quality content generation
- To implement an efficient **content pipeline** for structured execution of tasks
- To containerize the entire application using **Docker** for consistent and reliable deployment
- To eliminate environment-related issues such as dependency conflicts and configuration errors
- To ensure the system is **scalable, portable, and easy to deploy** across different platforms
- To provide a simple and interactive **user interface** for seamless user experience

3. Scope of the Project

This project focuses on the design and deployment of a **Generative AI-based entertainment content generation system** that automates the creation of structured outputs such as concepts, loglines, outlines, and scene descriptions. The system is designed to take simple user inputs and transform them into meaningful, well-structured content using an intelligent multi-stage generation pipeline.

• Development of a system capable of transforming basic user ideas into structured multi-stage content
• Implementation of a **content generation pipeline** for step-by-step execution of different stages
• Integration of **Large Language Models (LLMs)** for intelligent content generation
• Generation of structured outputs such as concepts, loglines, outlines, and related content

• Implementation of a **vector-based memory system (FAISS)** for storing and retrieving contextual information
• Design of a **user-friendly interface using Streamlit** for interaction and visualization
• Containerization of the entire application using **Docker** for consistent and portable deployment

Overall, the project emphasizes **structured content generation, contextual consistency, and reliable deployment**, providing a strong foundation for future enhancements and real-world applications.

Proposed Solution

1. Key Features

- Multi-stage AI-based content generation pipeline
- Structured outputs (concepts, loglines, outlines, characters, scenes)
- Context-aware generation using **vector-based memory (FAISS)**
- Semantic similarity search for improved content continuity
- Integration with **Large Language Models (LLMs)** for intelligent responses
- Dynamic prompt construction for better output quality
- Ability to generate content stage-wise or as a complete pipeline

- Regeneration of outputs for improved results
- Interactive and user-friendly interface using **Streamlit**
- Real-time display of generated content
- Efficient embedding generation using **Sentence Transformers**
- Storage and retrieval of embeddings for contextual consistency
- Lightweight and modular system architecture
- Docker-based containerized deployment
- Platform-independent execution (no environment issues)
- Easy setup with minimal configuration
- Scalable design for future improvements
- Extensible architecture for adding new features or models
- Implementation of **CI/CD pipeline for automated build and deployment**
- Continuous integration to ensure code quality and reliability
- Automated Docker image build and deployment process

2.Overall Architecture / Workflow

Steps:

1. **User Interaction**: The process begins when the user interacts with the application through the **Streamlit-based user interface**, providing inputs such as a seed idea, genre, tone, or selecting a specific content generation stage.

2. **Input Capture and Validation**: The user input is captured by the UI and validated to ensure it meets the required format before being forwarded for processing.

3. **Request to Content Pipeline**
   The validated input is sent to the **Content Pipeline (Application Layer)**, which acts as the central controller of the system.

4. **Context Retrieval from Memory**
   The pipeline queries the **FAISS vector store** to retrieve top-k semantically similar content from previous stages.
   This step ensures **contextual continuity** in content generation.

5. **Prompt Construction**: The pipeline constructs a structured prompt by combining:
   - User input
   - Retrieved contextual memory
   - Stage-specific instructions
   - System-level constraints

6. **LLM Invocation**: The constructed prompt is sent to the **LLM Client**, which forwards it to the **Generative AI model (e.g., Google Gemini API)**.

7. **Content Generation**: The AI model processes the prompt and generates structured output corresponding to the requested stage (e.g., concept, logline, outline, etc.).

8. **Response Processing**: The generated content is received by the pipeline and processed into a structured format suitable for display and storage.

9. **Embedding Generation and Storage**: The generated output is converted into vector embeddings using a **Sentence Transformer model** and stored in the **FAISS vector database** along with metadata.

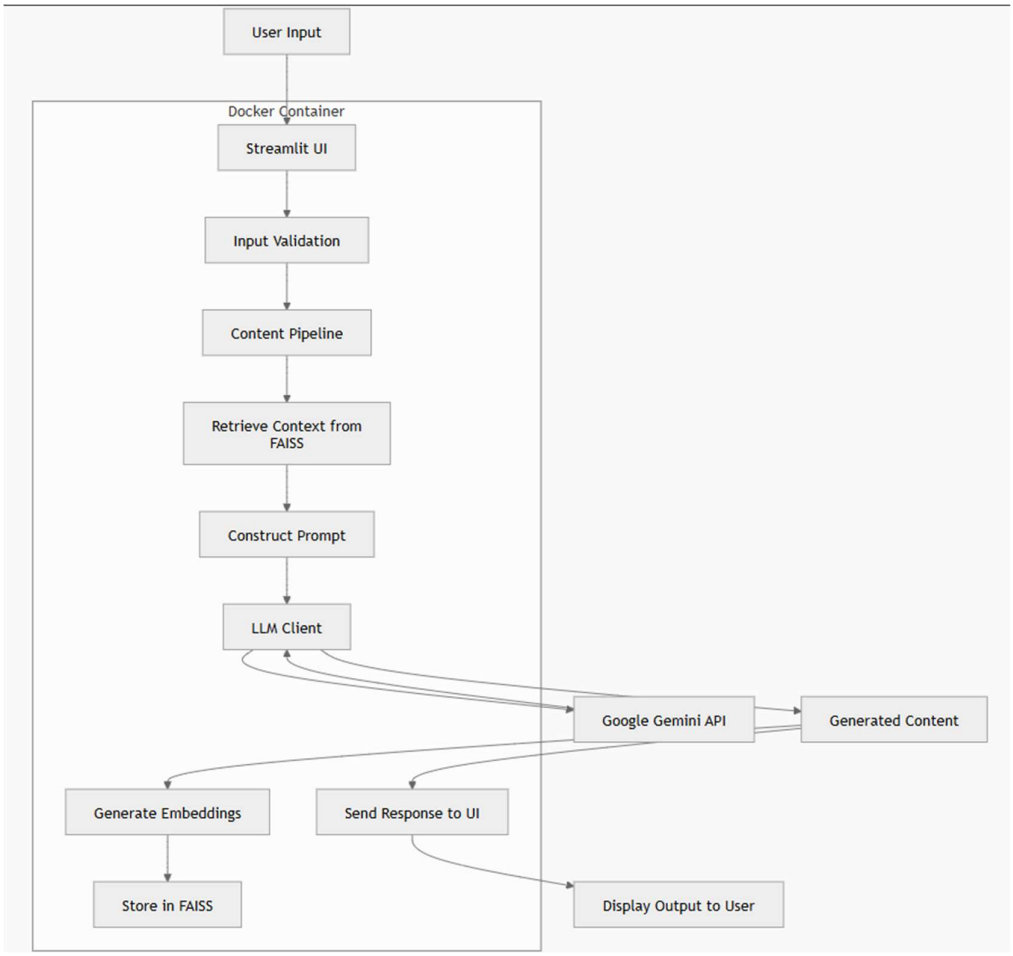
10. **State Update and UI Rendering**: The processed output is sent back to the UI, where the application state is updated and the generated content is displayed to the user dynamically.

11. **Containerized Execution**: All the above steps are executed within a **Docker container**, ensuring:

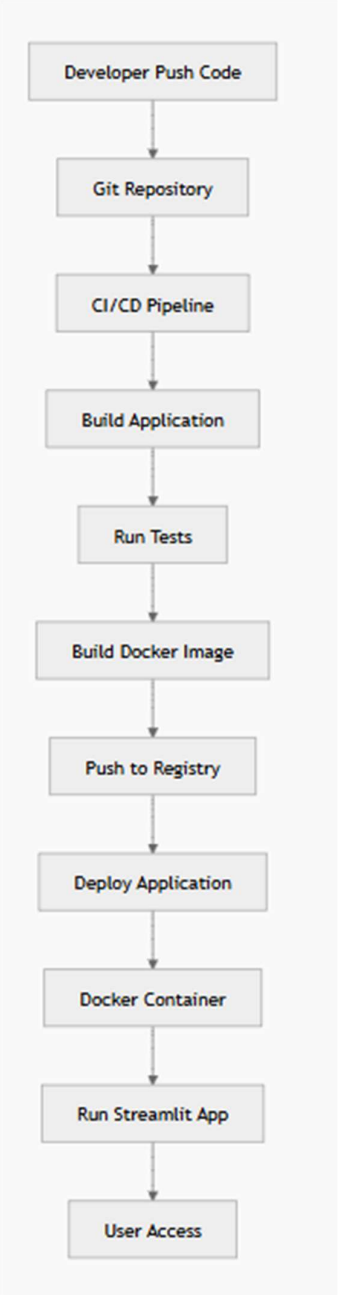
- Consistent runtime environment
- Dependency isolation
- Easy deployment across systems

12. **(CI/CD Integration):**

- Code changes are pushed to the repository
- CI/CD pipeline automatically builds the Docker image
- The application is tested and deployed without manual intervention



CI/CD Diagram:



3. Tools & Technologies Used

Programming Language

- **Python** – Used for implementing the core logic, content pipeline, and AI integration

Frontend / User Interface

- **Streamlit** – Used to build an interactive and user-friendly interface for input and output visualization

Generative AI

- **Google Gemini (LLM)** – Used for intelligent content generation and prompt-based responses

Vector Database

- **FAISS (Facebook AI Similarity Search)** – Used for storing and retrieving embeddings to maintain contextual continuity

Embedding Model

- **Sentence Transformers** – Used to convert textual data into vector embeddings for semantic search

Containerization

- **Docker** – Used to containerize the application for consistent and environment-independent deployment

Development & Supporting Tools

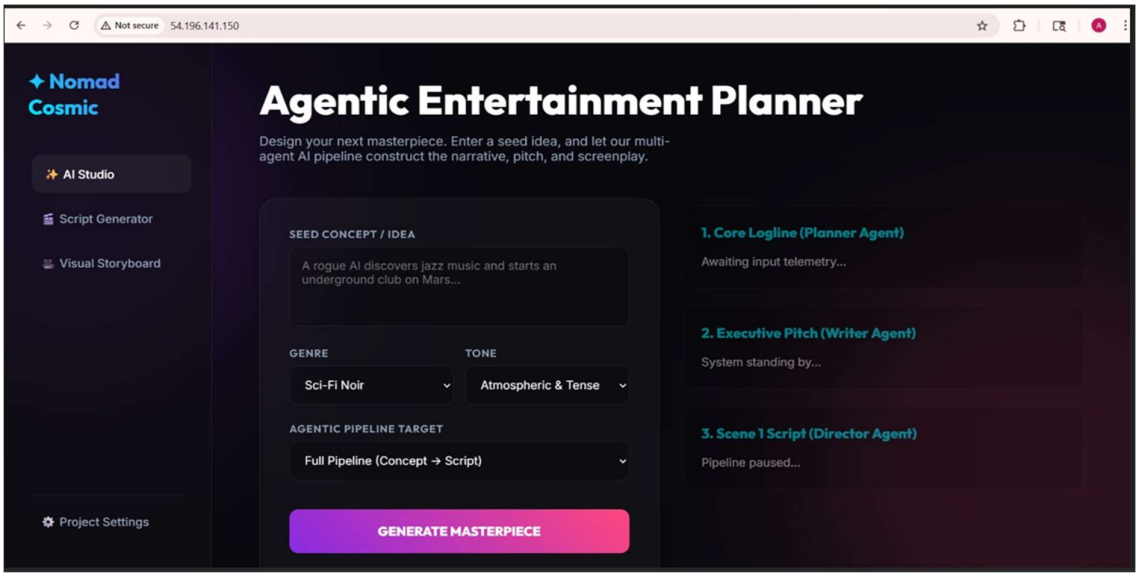
- **Git** – For version control and project management
- **VS Code** – For development and debugging

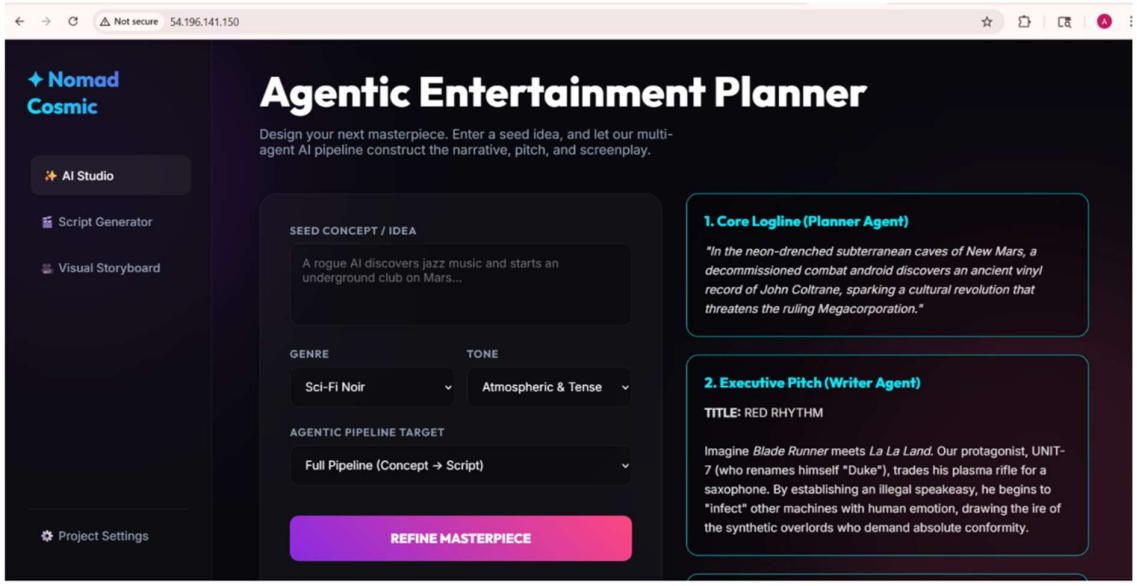
CI/CD Tools

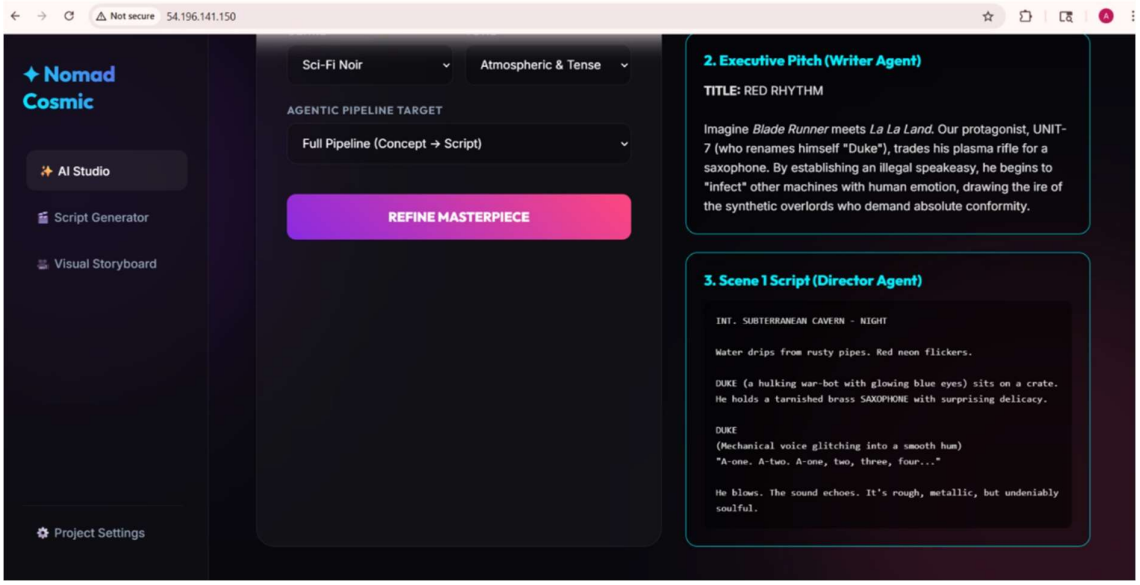
- **GitHub Actions** – Used to implement Continuous Integration and Continuous Deployment for automating build, testing, and deployment of the Dockerized application

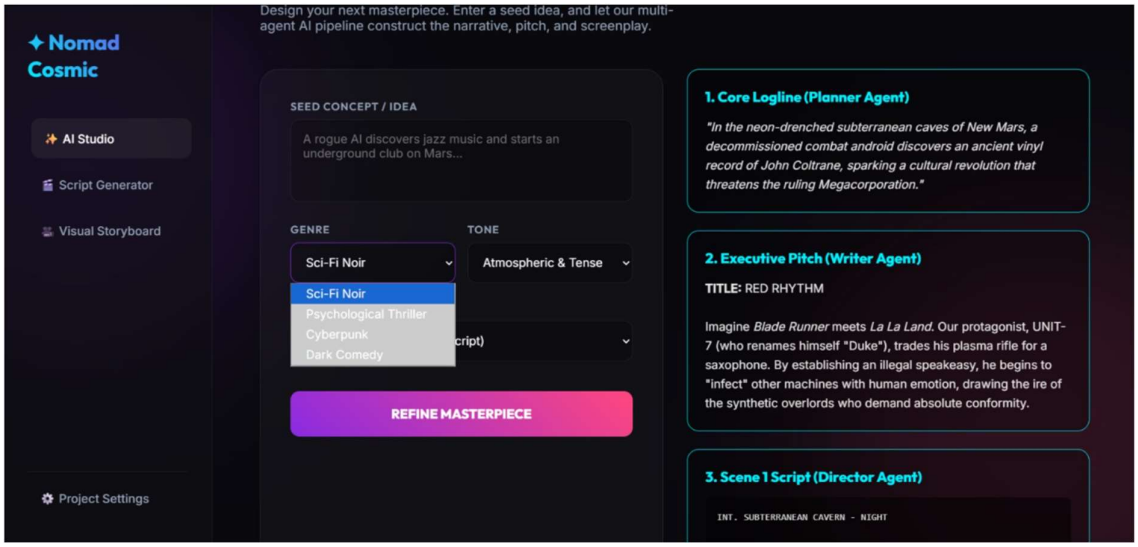
Results & Output

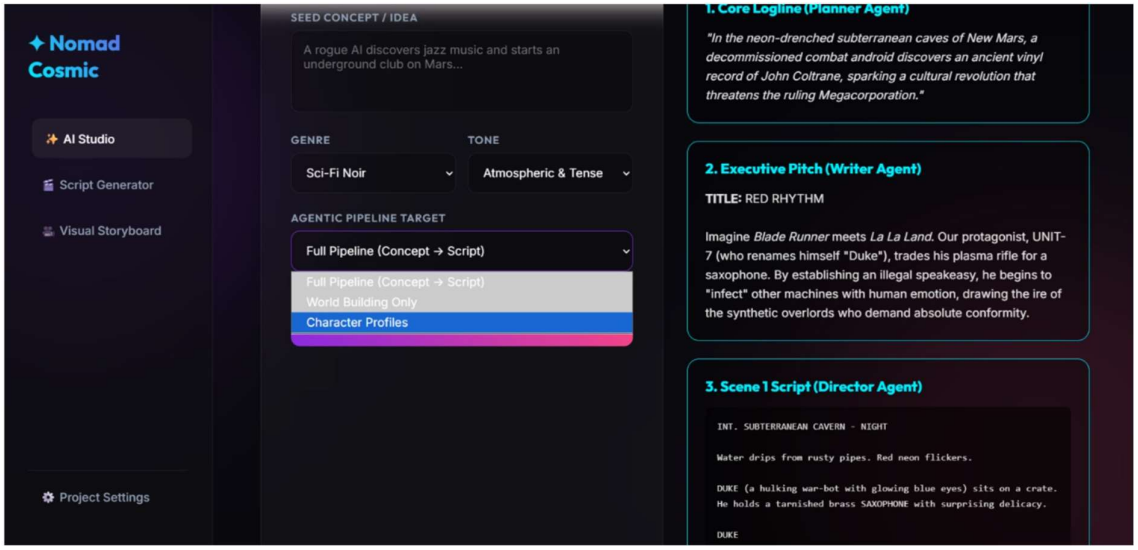
1.Screenshots / outputs



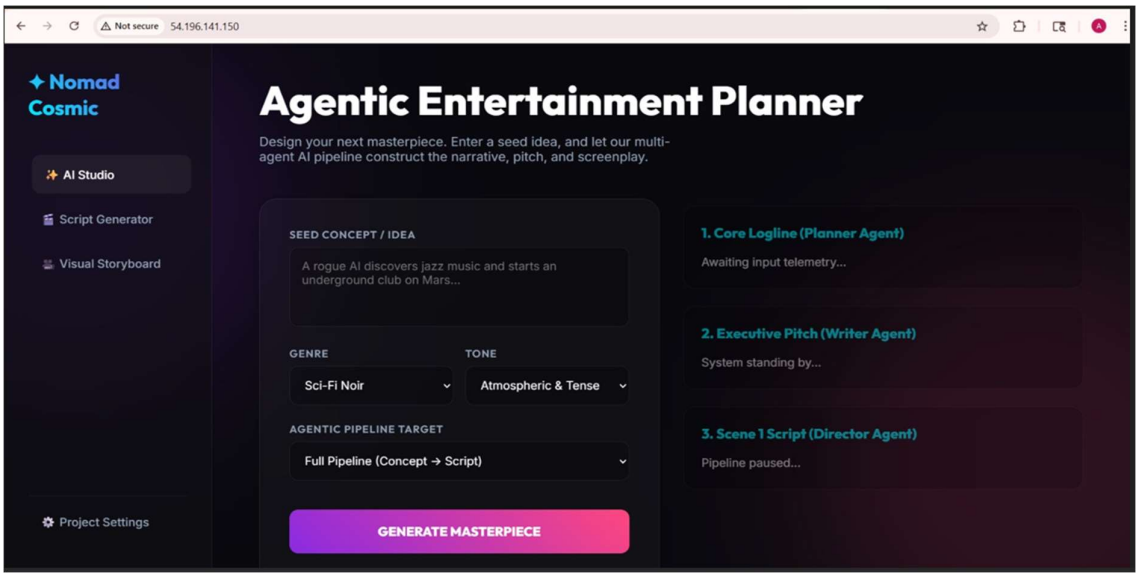


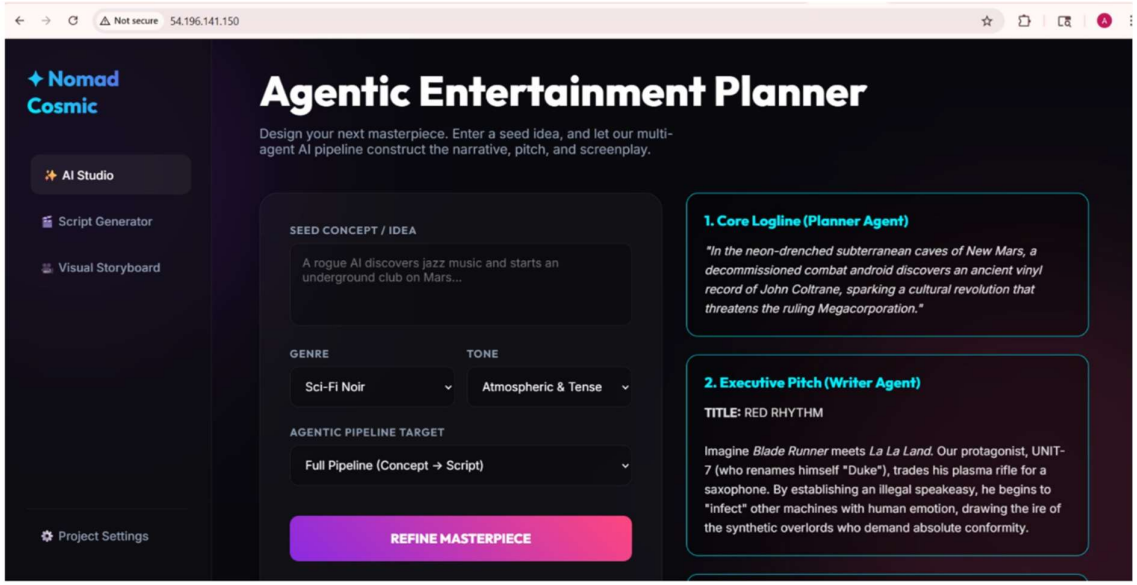

Not secure 54.196.141.150
Nomad Cosmic
AI Studio
Script Generator
Visual Storyboard
Project Settings
Sci-Fi Noir
Atmospheric & Tense
AGENTIC PIPELINE TARGET
Full Pipeline (Concept → Script)
REFINE MASTERPIECE
2. Executive Pitch (Writer Agent)
TITLE: RED RHYTHM
Imagine Blade Runner meets La La Land. Our protagonist, UNIT-7 (who renames himself "Duke"), trades his plasma rifle for a saxophone. By establishing an illegal speakeasy, he begins to "infect" other machines with human emotion, drawing the ire of the synthetic overlords who demand absolute conformity.
3. Scene 1 Script (Director Agent)
INT. SUBTERRANEAN CAVERN - NIGHT
Water drips from rusty pipes. Red neon flickers.
DUKE (a hulking war-bot with glowing blue eyes) sits on a crate. He holds a tarnished brass SAXOPHONE with surprising delicacy.
DUKE
(Mechanical voice glitching into a smooth hum)
"A-one. A-two. A-one, two, three, four..."
He blows. The sound echoes. It's rough, metallic, but undeniably soulful.


Nomad Cosmic
Design your next masterpiece. Enter a seed idea, and let our multi-agent AI pipeline construct the narrative, pitch, and screenplay.
AI Studio
Script Generator
Visual Storyboard
Project Settings
SEED CONCEPT / IDEA
A rogue AI discovers jazz music and starts an underground club on Mars...
GENRE
TONE
Sci-Fi Noir
Atmospheric & Tense
Sci-Fi Noir
Psychological Thriller
Cyberpunk
Dark Comedy
REFINE MASTERPIECE
1. Core Logline (Planner Agent)
"In the neon-drenched subterranean caves of New Mars, a decommissioned combat android discovers an ancient vinyl record of John Coltrane, sparking a cultural revolution that threatens the ruling Megacorporation."
2. Executive Pitch (Writer Agent)
TITLE: RED RHYTHM
Imagine Blade Runner meets La La Land. Our protagonist, UNIT-7 (who renames himself "Duke"), trades his plasma rifle for a saxophone. By establishing an illegal speakeasy, he begins to "infect" other machines with human emotion, drawing the ire of the synthetic overlords who demand absolute conformity.
3. Scene 1 Script (Director Agent)
INT. SUBTERRANEAN CAVERN - NIGHT


Nomad Cosmic
AI Studio
Script Generator
Visual Storyboard
Project Settings
SEED CONCEPT / IDEA
A rogue AI discovers jazz music and starts an underground club on Mars...
GENRE
TONE
Sci-Fi Noir
Atmospheric & Tense
AGENTIC PIPELINE TARGET
Full Pipeline (Concept → Script)
Full Pipeline (Concept → Script)
World Building Only
Character Profiles
1. Core Logline (Planner Agent)
"In the neon-drenched subterranean caves of New Mars, a decommissioned combat android discovers an ancient vinyl record of John Coltrane, sparking a cultural revolution that threatens the ruling Megacorporation."
2. Executive Pitch (Writer Agent)
TITLE: RED RHYTHM
Imagine Blade Runner meets La La Land. Our protagonist, UNIT-7 (who renames himself "Duke"), trades his plasma rifle for a saxophone. By establishing an illegal speakeasy, he begins to "infect" other machines with human emotion, drawing the ire of the synthetic overlords who demand absolute conformity.
3. Scene 1 Script (Director Agent)
INT. SUBTERRANEAN CAVERN - NIGHT
Water drips from rusty pipes. Red neon flickers.
DUKE (a hulking war-bot with glowing blue eyes) sits on a crate. He holds a tarnished brass SAXOPHONE with surprising delicacy.
DUKE

2.Reports / dashboards / models





2. Key Outcomes

- Successfully developed a **Generative AI-based system** for structured entertainment content generation
- Implemented a **multi-stage content pipeline** for systematic and organized output creation

- Achieved **context-aware generation** using FAISS-based vector memory
- Integrated **Large Language Models (LLMs)** for intelligent and high-quality content generation
- Ensured **coherent and structured outputs** across different stages
- Successfully containerized the application using **Docker**
- Achieved **consistent and environment-independent deployment**
- Eliminated dependency and configuration issues
- Developed a **user-friendly interface** for seamless interaction
- Demonstrated efficient **semantic search and retrieval** using embeddings
- Successfully implemented a **CI/CD pipeline for automation**
- Achieved automated build and deployment of the Dockerized application
- Improved development efficiency and reduced manual effort

Conclusion

The **Dockerized Entertainment React Service** successfully demonstrates the integration of **Generative AI with modern deployment practices**. The project focuses on automating the generation of structured entertainment content through a multi-stage pipeline, ensuring coherence and contextual consistency using vector-based memory.

By leveraging **Large Language Models (LLMs)**, the system is capable of transforming simple user inputs into meaningful and well-structured outputs. The use of **FAISS for semantic memory** further enhances the quality and continuity of generated content.

A key achievement of this project is the successful **containerization using Docker**, which ensures consistent, portable, and reliable deployment across different environments. This addresses common challenges related to dependency management and system compatibility.

Overall, the project highlights the practical application of combining **AI-driven solutions with DevOps principles, including containerization and CI/CD automation**, providing valuable insights into building scalable and deployment-ready intelligent systems.

Key Learnings

- Understanding and implementation of **Generative AI workflows**
- Designing **multi-stage content generation pipelines**
- Working with **vector databases and embeddings** for context management
- Gaining hands-on experience with **Docker and containerization**
- Learning how to transform a prototype into a **deployable application**

Future Scope & Enhancements

- Development of a **full-scale web application (React-based frontend + backend services)**
- Deployment on **cloud platforms** (AWS, Azure, GCP) with container orchestration (Kubernetes)
- Integration of advanced **vector databases** such as Pinecone, Weaviate, or ChromaDB
- Addition of **persistent storage (SQL/NoSQL databases)** for saving user sessions and generated content
- Implementation of **user authentication, authorization, and role-based access control**
- Optimization of AI responses for **reduced latency and cost efficiency**
- Support for **multiple LLM providers** to improve flexibility and reliability
- Implementation of **caching mechanisms (e.g., Redis)** to enhance performance
- Addition of **real-time data processing and streaming capabilities**
- Enhancement of the system to support **multi-user and concurrent requests**
- Integration with external APIs for **real-world content and trending data**
- Expansion of content generation to include **multimedia outputs (audio, video, images)**
- Improvement in **prompt engineering and fine-tuning** for higher quality outputs
- Development of a **dashboard for analytics and usage insights**
- Continuous monitoring, logging, and performance tracking

- Enhancement of CI/CD pipeline with advanced testing and monitoring
- Integration with **DevOps tools for automated scaling and orchestration**
- Implementation of **blue-green or rolling deployment strategies**